

Glance” later in this chapter, you probably won’t need them anymore. Big O is a simple concept and can be useful even if you don’t strictly follow mathematical rules.

Determining the Big O Order of Your Code

To determine the big O order for a piece of code, we must do four tasks: identify what the n is, count the steps in the code, drop the lower orders, and drop the coefficients.

For example, let’s find the big O of the following `readingList()` function:

```
def readingList(books):
    print('Here are the books I will read:')
    numberOfBooks = 0
    for book in books:
        print(book)
        numberOfBooks += 1
    print(numberOfBooks, 'books total.')
```

Recall that the n represents the size of the input data that the code works on. In functions, the n is almost always based on a parameter. The `readingList()` function’s only parameter is `books`, so the size of `books` seems like a good candidate for n , because the larger `books` is, the longer the function takes to run.

Next, let’s count the steps in this code. What counts as a STEP is somewhat vague, but a line of code is a good rule to follow. Loops will have as many steps as the number of iterations multiplied by the lines of code in the loop. To see what I mean, here are the counts for the code inside the `readingList()` function:

```
def readingList(books):
    print('Here are the books I will read:') # 1 step
    numberOfBooks = 0                         # 1 step
    for book in books:                       # n * steps in the loop
        print(book)                         # 1 step
        numberOfBooks += 1                  # 1 step
    print(numberOfBooks, 'books total.')
```
